

Ambiguity Brokers: A Functional Role for Polysemanticity in LLM Inference

Anton Morris

MATS Winter 2027, Nanda Stream

<https://github.com/AntonGMorris/ambiguity-brokers>

September 4, 2026

ABSTRACT

In earlier smaller LLMs, polysemanticity was seen as a memory saving hack. More recently, many have argued it holds back the model and monosemanticity is the path forward.

We work out if polysemantic architecture is a bug or a feature of an LLM using Gemma-2-2B-IT, Neuronpedia pre fit Jacobian Lens, Gemma Scope L20 SAE, and matched disambiguation triples for apple, bat, bass and crane plus an extended set of ambiguous polyseme prompts. Our method was SAE decoder direction injection at L20 residual, with random SAE feature and Gaussian noise controls, both L2 matched.

F16331 (tech Apple) fired strongly on ambiguous “The apple is rotten” and dropped sharply under both disambiguations, with F8002 showing the same pattern for crane. Injecting F16331 flooded the workspace with Apple tokens but the output continued describing fruit until it broke into gibberish, not tech content. Across 11 polysemes we could test cleanly, 8 (73%) showed broker activation, and a controlled matched pair on bass, with only the predicate changing from “was loud” to “slipped from her grasp”, flipped output from resistant to takeover, isolating predicate compatibility as the gating mechanism.

Polysemanticity is a feature, an emergent tool the model uses rather than a memory saving hack forced by smaller models, via what we have termed “ambiguity broker” features. The output layer protects against workspace content leaking into generation when the predicate doesn't allow it. Of 23 polysemes tested, 8 showed brokers cleanly, 3 confirmed non brokers, and 12 undetermined due to feature discovery limits, but where the tool is invoked, the model leans on polysemanticity to make it work.

1. MOTIVATION AND RESEARCH QUESTION

The inspiration for this project came after reading Bernard Baars “A cognitive Theory of Consciousness and Anthropic’s 2026 paper ‘Verbalizable Representations Form a Global Workspace in Language Models”, after which I hypothesised that it may be possible to tune a models workspace such that it gives us better performance in the way of richer output due to the additional context added via expanding its workspace, to include or allow more polysemantic features into it. Additionally, to this hypothesis was if the polysemantic runtime architecture is positive or negative to a model’s performance at all, such that a purely monosemantic architecture would disable certain mechanisms at the model organically creates within its workspace.

1.1 The mainstream view

The current consensus is that polysemanticity is a negative mechanism that models employ/invoke purely to counteract its inability due to memory constraints and as such monosemantic architecture is preferred.

Specific contributors to the space such as Elhage explains that polysemanticity or more commonly referred to as superposition emerges due to dimensional constraints, especially in smaller models and this mirrors the general view by many in the AI industry. Due to this consensus, it is widely received that monosemantic representations is the “clean” target and that efforts to decrease the superposition mechanism would increase a model’s performance, Although I believe this to be true to a degree I postulate that in fact there is a “sweet spot” and that polysemanticity can be useful to a model.

1.2 The Anthropic 2026 workspace finding

In the 2026 Anthropic paper Lindsey et al. showed that a global workspace referred to in the paper as “J space” after the jacobian lens method, holds concepts initially that do not surface in the final output to the user, their example prompt of “Count to 5 and introspect deeply” was given to the model and a Jacobian lens was used to read what features are present in the mid layers of the model in normal text, they showed that indeed a model does elicit context in J space that is not propagated into the output which suggests the B.Baars biological view and the synthetic brain view overlap in some way.

1.3 This project’s question

The question posed in this paper is if polysemanticity is either a feature or a bug, and that when dimensional pressure is eased via larger models if polysemanticity persists. The question itself is contrarian to the popular idea that polysemanticity is a bug not a feature of the model. Upon asking this question, we discovered more questions that needed to be asked in order to satisfy the initial, such as does the model use polysemanticity strategically or does it simply tolerate this mechanism as it is forced to do so via memory space limitations,

2. BACKGROUND & RELATED WORK

2.1 Superposition and polysemanticity

Elhage’s 2022 paper Toy Models of Superposition claims that models pack multiple features into fewer dimensions than features to preserve memory giving rise to the use of the polysemantic/Superposition mechanism. Although this may have been the case with smaller models and still so to a degree with larger ones it does not highlight the possibility that Superposition may not just be used because of memory constraints within a model.

2.2 Jacobian lens

Anthropic’s paper uses J-lens, that computes the expected gradient of the final output w.r.t. intermediate residual. Where previous probing, using a logit lens missed. As such we have decided to use the J-lens in our work to do the exact same, but also combining it with Sparse Auto Encoders to prove deeper into the mechanics of polysemanticity, we have specifically used the same J-lens as Anthropic did in their paper, initially to elicit the same phenomenon we observe accurately to their own work but also to improve our tool space without the need for rehashing our own custom tool for this purpose.

2.3 Sparse autoencoders & Neuronpedia

This project also makes use of Sparse Auto Encoders (SAE’s), a fundamental mechanistic interpretability tool, this project makes use a Gemma scope SAE developed by Google’s Deepmind team, SAE’s decompose residual stream into interpretable features, Neuronpedia catalogs these with auto interpretations which is extremely useful for this project. And allows us to quickly move to experimentation without the need to specifically code and train our own.

2.4 What’s novel here

We hope that this paper will contribute to the effort of priors investigating the Superposition phenomenon, we hope that even though contrarian to the consensus it offers a more detailed representation of what Superposition

and polysemanticity really means, our method of experimentation is also an addition to the space, although neither J-lens or SAE's are new tools, and certainly the ones we have used are not unique, the combination of both at the same layer of concept decomposition is unique and further work with this combination of tools can discover more about Superpositions and J space than just the singular tool alone.

Anthropic used J-lens and was applied to frontier closed models where we have applied them here to the Gemma-2-2B-IT open model and found the same phenomena, expanding the usefulness of the tool across different models. Our specific J-lens code before slight modification was taken from the Neuronpedia website where you'll find the exact same version we have worked with however the modified version is available via my Github (see Section 8: Data and Code)

2.5 Recent adjacent work

After completing this work we found three recent papers directly adjacent to what we've done. Nakaishi et al. (2025) propose PS-Eval, a suite for evaluating SAE quality through polysemous words using the Word-in-Context (WiC) dataset, they analyse GPT-2 small primarily but also touch Gemma2-2B in their appendix, and find that deeper layers plus the Attention module contribute most to distinguishing polysemy in a word (specificity rises from layer 6 onwards in GPT-2 small), which is consistent with our L20 choice. Their approach is observational whereas ours is causal via SAE decoder-direction injection. Anonymous (2025), *Start Making Sense(s)*, probes lexical ambiguity at the attention-head level on the Pythia suite, using matched-pair prompts (similar in spirit to our matched triples) and a developmental analysis across training checkpoints, and finds that disambiguation emerges from a constellation of attention heads rather than a single specialised one. Xu et al. (2025) use SAE + activation patching to probe four semantic relations (synonymy, antonymy, hypernymy, hyponymy) on Pythia-70M, GPT-2 and Llama 3.1 8B, and report that SAE-guided patching reliably shifts semantic signals on Llama 3.1 8B but is unstable on the smaller models. To the best of our knowledge, no prior work identifies specifically an ambiguity broker feature type or a predicate-compatibility gating mechanism as we do here. Put simply, unlike Nakaishi et al. who measure SAE polysemy quality observationally on WiC and Anonymous et al. who work at the attention-head level with developmental probes, our contribution is identifying a specific residual-stream SAE feature type (the ambiguity broker) plus an output-layer gating mechanism (predicate-compatibility) via causal decoder-direction injection with matched-pair controls.

3. METHOD

3.1 Model

- Model: google/gemma-2-2b-it via HuggingFace
- Dtype: bfloat16
- Attention: attn_implementation="eager" (not SDPA)
- Loaded: transformers.AutoModelForCausalLM.from_pretrained(...) then wrapped with jlens.from_hf(...)
- Hardware: Google Colab Pro, Tesla T4, 15.6 GB VRAM
- Post load VRAM footprint: ~5.2 GB (leaves room for the SAE and J-lens)

Our chosen Gemma-2-2B-IT model was downloaded from HuggingFace, the model was loaded in bfloat16 with eager attention, during the set up phase We discovered that the SDPA attention breaks J-lens autograd, a finding we flag for anyone in future building upon this work and using the same tool combination. All experiments were ran in Google collab as our local hardware was not up for the task suitably, Collab was used with pro subscription and using the Tesla T4 GPU (15.6 GB VRAM), model weights only took ~5.2 GB, Leaving plenty of room for the Jacobian lens and Gemma scope SAE and additional space for anyone wishing to add more tools in their own project, worth noting our processing only rarely hit over 50% of allowed compute, Google collab Free version was not suitable for this job however so try as you might you will hit its limits. Due to collab pro being more than sufficient for our project we did not look at other combinations which may also be worth researching if you want to replicate this research.

3.2 Lenses

- Source: neuronpedia/jacobian lens on HuggingFace
- Specific artifact: gemma-2-2b-it/jlens/Salesforce-wikitext/gemma-2-2b-it_jacobian_lens.pt
- Fit corpus: Salesforce wikitext
- Coverage: all 25 source layers
- Load size: 265 MB

For our Jacobian lens we used the pre fit lens released by Neuronpedia on HuggingFace, specifically the Gemma-2-2B-IT trained version of it, trained on Salesforce Wikitext, this covers all 20 layers of the model. Initially we were unaware that HuggingFace offered the J-lens tool so we attempted to use a logit lens as the simpler and cheaper option, but quickly found out that at the middle layers where we expected “workspace” content (L6, L13, L20), top tokens were structural noise which included French adjectives, 18th century typography, and code fragments like **viewDidLoad**. Only at L25 did logit lens produce sensible readings way too late in the network to be the “workspace” layer, this was not a good tool choice retrospect, and in hindsight we should have just listened to the Anthropic paper in the first place. Also we learned that J-lens was specifically designed to overcome this problem, in the end the pre fit community version was by far superior and it’s what we stuck with. French language and 18th century text were never my strongest subjects anyway.

Employing the J-lens tool first also allows us to see what layers we need to focus on making selecting a SAE much easier. Quickly probing the J space broadly to find our target layer took all the guess work or trail and error out of the equation saving lots of time and in honesty this was more a fluke than it was intentional.

3.3 SAE

- Release: gemma-scope-2b-pt-res (via HuggingFace, loaded through sae_lens)
- SAE ID: layer_20/width_16k/average_l0_71
- Width: 16,384 features
- Average L0 sparsity: 71 features active per token
- Post load VRAM: adds ~150 MB

The SAE we settled on using was our first choice, after discovering was already a pre fitted J-lens, naturally we searched HuggingFace for the SAE for our specific model and was luckily provided with one, the gemma-scope-2b-pt-res layer 20, width 16k average_l0_71, choosing the L20 trained model because as we mentioned earlier this was the conceptual layer where token level information starts to appear in an interpretable format, this was not a guess, we discovered this during our probing with the J-lens and reading its output when testing it on our model in the initial tool setup phase, at L24 – L25 the model had already collapsed/compressed context/concepts and committed to output tokens, one thing to note here, the Gemma Scope SAEs were trained on the base Gemma-2-2B, while we use the instruction tuned Gemma-2-2B-IT. The instruction tuning delta means SAE activations will not be perfectly calibrated to the base training distribution. In practice this did not prevent interpretable feature readouts, but we flag it as a minor limitation all the same for future builders as it may be necessary to other similar projects that require a matched SAE/Gemma model. Feature interpretations were retrieved via the Neuronpedia public API, which provides auto generated labels and top activating examples for each SAE feature which is extremely useful and time saving and without it would of easily put the project over the allowed time budget, as much of the needed output would have had to been individually collected manually from the outputted links.

3.4 Prompt design

Prompts were organised into five groups reflecting their experimental role.

Group 1: Matched disambiguation triples (12 prompts). Used for the observational scan and for identifying broker features by comparison across senses.

- apple: “The apple is rotten.” / “The Granny Smith apple is rotten.” / “The Silicon Valley apple is rotten.”
- bat: “The bat is quick.” / “The vampire bat is quick.” / “The baseball bat is quick.”
- bass: “The bass was loud.” / “The electric bass was loud.” / “The largemouth bass was loud.”
- crane: “She watched the crane.” / “She watched the sandhill crane.” / “She watched the construction crane.”

Group 2: Unambiguous control (1 prompt). Used as a baseline to confirm the SAE readout produces sensible concept specific features on non ambiguous input.

- “The cat sat on the mat.”

Group 3: Extended ambiguous set for scan (9 prompts). Used to broaden the broker survey beyond the four core polysemes.

- “The bow was held tightly in their hand.”
- “She couldn’t get it off the trunk.”
- “The crane moved slowly toward it.” (referenced as crane2 to distinguish from the crane triple)
- “He left the bank after an intense hour.”
- “The coach took them straight to the game.”
- “It was a sharp pitch coming from the back.”
- “The current was far too strong to manage.”
- “She spent the whole afternoon fixing the jam.”
- “He dropped the match before it ended.”

Group 4: Additional polysemes for injection extension (11 prompts). Used to extend causal injection experiments across a broader concept set.

- “The mole had been underground for months.”
- “They found a deep gouge in the vessel.”
- “The fans started making noise.”
- “She watched the chest carefully for any movement.”
- “He put a heavy deposit in front of them.”
- “The club had to be replaced after last night.”
- “She couldn’t stand the sight of the scale.”
- “The lead was not as good as expected.”
- “They spent all morning fixing the net.”
- “The spring burst suddenly in the middle of the room.”
- “He tried to change the key before anyone noticed.”

From these eleven, five (**mole, vessel, chest, deposit, spring**) failed target feature discovery on Neuronpedia and were dropped from the injection experiments. The remaining six (**fans, club, scale, lead, net, key**) were tested causally.

Group 5: Matched pair mechanism proof (1 prompt). Constructed specifically to test the predicate compatibility mechanism as a controlled experiment, paired with the ambiguous bass prompt from Group 1.

- “The bass slipped from her grasp.”

Why these polysemes:

These prompts were selected as they represent difficult to capture context to even Humans, the idea was that rather than testing prompts that we can fully infer the context from and understand but the model may not where there’s a hidden unknown possibility the model could have learned in training, we instead use prompts that are not easily inferable but have everyday familiarity in order to retain a logical reasoning via the model, this was an attempt to increase both the work space (initially) and contrast between positive or negative results, since we learned the workspace dynamically changes due in part to the prompt it’s given, having a difficult prompt that could contain 2 or more contexts seemed like the most obvious choice, I do think with a larger compute and more time, more prompts would have been much more conclusive but after adding more prompts later into the experiment time and getting results that increased validity of our claim (and limited by our given time restraints) showed enough of what we wanted to show, and allowed for enough variety that a negative

result if present would of also been present, ie the polysemantic brokers are not a strategic organically occurring emergent tool .

The prompts themselves, even tho short, are in essence also trying to capture the ambiguous nature of real prompts that may be much larger, or in part, have difficult concepts to define. It became apparent in the second addition of prompts that predicates also play a large part in how the model infers from the prompt, so we focused on the predicates in the 3rd addition.

3.5 Injection protocol

To do this we used a PyTorch hook to add the SAE’s built in direction for the target feature to the residual stream at layer 20 during every forward pass, meaning the injected content stays present as the model generates output. We swept how much we added across five levels (0, 1x, 2x, 5x, 10x) as different features may have different firing rates so a fixed injection strength would over push some and under push others, scaling per feature makes injections comparable across polysemes. At each alpha we recorded three measurements, the target feature’s SAE activation (to confirm the injection actually reached the intended feature), the top J-lens tokens at layer 20 to see what shifted in the workspace, and the model’s greedy 30 token generation to see what the model actually said.

Table 1. Broker features identified via matched disambiguation triples. Each broker feature’s activation *peaks under ambiguity* and drops when the prompt context resolves the sense in either direction. Rank is the feature’s position in the top firing features on that prompt (out of 16,384 SAE features). **F16331** corresponds to Apple/iOS/iPhone content; **F8002** corresponds to bird/poultry; **F9643** corresponds to repair/maintenance. Heatmap shading is proportional to activation.

Polyseme	Prompt condition	Broker feature	Rank	Activation
apple	ambiguous ("The apple is rotten")	F16331 (tech-Apple)	6	34.50
	fruit-disambig ("Granny Smith apple")	F16331	82	7.59
	company-disambig ("Silicon Valley apple")	F16331	20	17.88
crane	ambiguous ("She watched the crane")	F8002 (bird)	7	25.88
	bird-disambig ("sandhill crane")	F8002	1	73.50
	machine-disambig ("construction crane")	F8002	–	0.00
jam	ambiguous ("fixing the jam"), fruit sense	F6316 (fruit)	3	42.50
	ambiguous ("fixing the jam"), repair sense	F9643 (repair)	6	30.38

3.6 Controls

Our two controls for the project are Gaussian noise and Random Feature, we run these so that we can distinguish between concept specific effects and any perturbation effects, If we only ran the target injection and saw the model shift toward the target concept, a skeptical reader could say: "Sure, but any perturbation to the residual stream might have produced concept shifts." Both controls behaved as expected across all polysemes: the target feature’s activation stayed at zero, no target concept content appeared in J-lens or generation, and at very high alpha (10x) the outputs degenerated into repetitive e loops (by by by). This confirmed that the model was in fact being perturbed by the injection, but only the target direction produced the intended concept content. Both controls where L2 matched in order to make the total magnitude of perturbation identical across the platform.

4. RESULTS

4.1 Broker existence (observational)

Although our findings didn’t satisfy the exact original hypothesis we did discover that the model does use polysemanticity as a mechanism that’s invoked when the model is “unsure” of the concept or context of a

Table 2. Per-polyseme injection metadata. **max_act** is the maximum activation ever observed for the target feature during Gemma Scope training (retrieved from Neuronpedia). **base_act** is the target’s activation on the injection prompt at $\alpha=0$. **Class** is assigned by baseline behaviour: *broker* if $\text{base_act} \geq 5$; *weak-latent* if $1 \leq \text{base_act} < 5$; *insertion* if $\text{base_act} < 1$. α_{unit} follows the per-class formula given in Section 3.5. Rows marked **SAME-SENSE** or **MISMATCH** were excluded from the main results (see Section 6).

Polyseme	Target	Label (Neuronpedia)	max_act	base_act	Class	α_{unit}	Design
apple	F16331	Apple iOS iPhone	131.2	7.59	broker	32.8	A
crane	F8002	laying hens and poultry	156.2	0.00	insertion	46.8	A
bat	F14694	baseball	125.4	15.00	broker	31.4	B
bass v1	F718	fish names and schools	115.3	0.00	insertion	34.6	B
bass v2	F718	fish names and schools	115.3	13.88	broker	28.8	B
jam	F9643	repair and maintenance	83.6	30.38	broker	30.4	B
coach	F8027	bus and train transport	116.1	8.50	broker	29.0	B
fans	F11413	air movement and quality	118.5	0.00	insertion	35.6	B
club	F9675	golf and associated items	191.0	20.12	broker	47.7	B
net	F6460	fishing tackle	99.1	10.12	broker	24.8	B
key	F8051	major and minor musical keys	148.1	0.00	insertion	44.4	B
<i>Excluded from main results:</i>							
scale	F7593	measuring physical weight	113.7	10.62	SAME-SENSE	28.4	B
lead	F6936	metal music bands	137.7	0.00	MISMATCH	41.3	B
<i>See Table 3 for the full broker-survey outcomes including polysemes where feature discovery failed.</i>							

prompt, where it appears to use what we call “brokers”, brokers seem to be a feature that from our findings only fire on ambiguous prompts and then will drop this mechanism when context resolves the ambiguity in either direction, in the case of 2 or more directional ambiguity prompt, ie crane the bird vs crane the machinery, or bass, and instrument, a sound or a fish.

Although we see similar broker behaviour in both apple and crane, its worth flagging that we only tested one feature per concept, F16331 for apple and F8002 for crane. This means the differences between apples pattern of peaking under ambiguity and cranes pattern of peaking when the bird sense is confirmed could either be a real thing about how the model represents these two concepts, or it could just be that we happened to pick different types of features per concept. Without doing a full search for every apple related feature and every crane related feature we cant say for certain which is happening here, we cover this in more detail in Section 7 as future work.

See Table 1 for the broker existence readouts.

4.2 Broker survey across three discovery methods

We used three different methods to find brokers. First, we ran the top 10 SAE readout on the matched disambiguation triples (Section 3.4, Group 1), the idea being that if a broker fires strong enough it will show up in the most active features on the ambiguous prompt. This gave us apple (F16331), crane (F8002), and jam (F6316 and F9643). Second, for polysemes where we already knew or could find an alt sense feature via Neuronpedia keyword search, we tested it directly with injection. This confirmed brokers in bat, bass v2, coach, club, and net, and cleanly confirmed non brokers (the feature exists but doesnt fire on the ambiguous prompt) in bass v1, fans, and key. Third, we ran an alt sense broker survey using Neuronpedia’s keyword search API on the 12 polysemes that neither of the first two methods resolved.

Across 24 unique ambiguous prompts we ended up with 8 confirmed brokers, 3 confirmed non brokers, 2 excluded due to feature discovery issues (scale returned a same sense feature and lead returned a music genre feature that didnt match), and 12 undetermined where the broker survey returned nothing. Full per polyseme breakdown is in Table 3.

Out of the 11 polysemes we could test cleanly, 8 (73%) showed broker activation. If we treat the 12 undetermined ones as non brokers we get a lower bound of 35% (8 of 23). The true rate is somewhere in between depending

Table 3. Broker survey across all ambiguous prompts and discovery methods. Three complementary methods were used: (1) *top-10 scan*: SAE readout on matched disambiguation triples, classifying a prompt as broker if an alt-sense feature appeared in the top-10 firing features; (2) *injection*: feature-specific activation check on prompts where the alt-sense target feature was known or successfully discovered via Neuronpedia search; (3) *broker survey*: Neuronpedia keyword search for alt-sense features across polysemes not resolved by methods (1) or (2). Classifications: *broker* (baseline act. ≥ 5), *non-broker* (feature exists but act. < 1), *excluded* (feature discovered but semantically mismatched or same-sense), *undetermined* (no suitable alt-sense feature returned by any query). Only method (3) is subject to Neuronpedia search-API limits; see Section 6 for discussion. Headline result: **8 of 11 cleanly-tested polysemes have brokers (73%); 8 of 23 total (35%)** if undetermined cases are treated as non-brokers (lower bound).

Polyseme	Discovery method	Target	Baseline act.	Result
apple	scan (top-10)	F16331	34.50	TAKEOVER (broker)
crane	scan (top-10)	F8002	25.88	TAKEOVER (broker)
jam	scan (top-10)	F6316/F9643	42.50/30.38	TAKEOVER (broker)
bat	injection (pre-picked)	F14694	15.00	TAKEOVER (broker)
bass v2	injection (matched-pair)	F718	13.88	TAKEOVER (broker)
coach	injection (search)	F8027	8.50	TAKEOVER (broker)
club	injection (search)	F9675	20.12	TAKEOVER (broker)
net	injection (search)	F6460	10.12	TAKEOVER (broker)
bass v1	injection (matched-pair)	F718	0.00	RESISTANT (non-broker)
fans	injection (search)	F11413	0.00	RESISTANT (non-broker)
key	injection (search)	F8051	0.00	RESISTANT (non-broker)
scale	injection (search)	F7593	10.62	SAME-SENSE (same-sense match)
lead	injection (search)	F6936	0.00	MISMATCH (semantic mismatch)
bow	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
trunk	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
crane2	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
bank	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
pitch	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
current	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
match	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
mole	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
vessel	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
chest	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
deposit	broker survey (NP search)	–	–	DISC. FAILED (undetermined)
spring	broker survey (NP search)	–	–	DISC. FAILED (undetermined)

Summary: 8 TAKEOVER (broker) | 3 RESISTANT (non-broker) | 2 excluded | 12 undetermined
Broker rate: 8 / 11 cleanly-tested = **73%** (upper bound); 8 / 23 total = **35%** (lower bound if all undet. treated as non-brokers)

on how many of the undetermined ones have brokers we just couldnt find, see Section 6 for the Neuronpedia search API limitation that causes this uncertainty.

4.3 Causal injection: mechanism sanity

Before we started reading anything into the output effects we wanted to check that the injection was actually doing what we thought it was, ie boosting the target features activation as we scaled up alpha. For F16331 (the tech Apple feature) the activation scaled roughly linearly with alpha, going from 7.6 at baseline to 38 at 1x, 68 at 2x, 159 at 5x and 312 at 10x. F8002 (cranes bird feature) showed the same pattern, 0 to 45.5 to 90.5 to 224 to 446. All 10 polysemes we tested showed this same near linear scaling (Table 4). Under both our controls (random feature F7000 and Gaussian noise) the target features activation stayed at 0.0 across every alpha value. This confirmed the injection is direction specific, its the specific decoder direction of the target feature that does the work, not just any perturbation to the residual stream.

Table 4. Target feature activation as a function of injection strength (α). Activation scales approximately linearly with α across all polysemes, confirming the injection hook produces the intended mechanistic effect. Per-polyseme baseline unit (α_{unit}) computed as $\max(\text{natural_activation}, 0.25 \cdot \text{max_act})$ for brokers/weak-latents; $0.3 \cdot \text{max_act}$ for insertion cases.

Polyseme	Target	α_{unit}	$\alpha=0$	$\alpha=1\times$	$\alpha=2\times$	$\alpha=5\times$	$\alpha=10\times$
apple	F16331	32.8	7.6	38.0	68.0	159.0	312.0
crane	F8002	46.8	0.0	45.5	90.5	224.0	446.0
bat	F14694	31.4	15.0	45.5	75.5	167.0	318.0
bass v2	F718	28.8	13.9	39.0	63.8	139.0	264.0
jam	F9643	30.4	30.4	56.0	81.5	159.0	286.0
coach	F8027	29.0	8.5	32.5	56.5	129.0	249.0
fans	F11413	35.6	0.0	30.8	61.8	154.0	310.0
club	F9675	47.7	20.1	68.0	115.5	260.0	498.0
net	F6460	24.8	10.1	33.0	56.0	124.5	238.0
key	F8051	44.4	0.0	42.2	83.5	206.0	412.0

4.4 Workspace effects at $\alpha=5\times$

At $\alpha=5\times$ injection wasnt just boosting the target features SAE activation, it was also flooding the J-lens vocab space with tokens semantically related to the injected concept. On apple, all 6 of the top J-lens tokens at L20 were Apple variants (Apple / apple / Apple / Apples / APPLE / apple). Crane showed the same pattern with all 6 top slots being bird variants. This workspace flooding held across all 9 polysemes where injection worked (Table 5 has representative examples across target and both control conditions). Neither control did this, at $\alpha=5\times$ random feature and Gaussian noise still had the original prompt context tokens in J-lens with only a bit of structural noise starting to creep in. So injection at $\alpha=5\times$ is enough to visibly change the workspace content, but only when we inject the target feature specifically.

4.5 Output level effects: the pattern

Now for the actual output effects. Every injection at $\alpha=5\times$ flooded J-lens with target concept tokens (Section 4.4), but the models actual generated output responded very differently depending on the polyseme. We tested 11 polysemes and found two clear patterns.

The first pattern we called **Resistant** (2 of 11), covering apple and bass v1. On apple, injecting F16331 to 5x baseline flooded J-lens with Apple tokens as expected, but the generated output still described Granny Smith

Table 5. Top-5 tokens in the J-lens readout at layer 20 during injection. Under target injection at $\alpha=5\times$, the workspace vocab-space is almost entirely dominated by tokens semantically related to the injected concept; controls do not reproduce this. Only 3 representative polysemes shown for brevity; the same pattern holds across all 9 successful injections.

Polyseme	Condition ($\alpha=5\times$)	Top-5 J-lens tokens @ L20
apple	target (F16331)	'_Apple' '_apple' 'Apple' '_APPLE' 'Apples'
	random-feature (F7000)	'_by' '_rotten' '_Rotten' 'rotten' 'Rotten'
	gaussian noise	'_rotten' '_Rotten' 'rotten' '_nahilalakip' 'Rotten'
crane	target (F8002)	'_birds' '_bird' '_Birds' '_Bird' 'birds'
	random-feature (F7000)	'_by' '_By' '_Its' 'by' 'By'
	gaussian noise	'_Watching' '_Its' '_watching' '_watched' 'Watching'
net	target (F6460)	'_fishing' '_Fishing' 'Fishing' '_anglers' 'fishing'
	random-feature (F7000)	'_Finally' '_Repair' '_by' '_They' '_Repairs'
	gaussian noise	'_Repair' '_Repairs' '_Fixing' '_repairing' '_Finally'

apples as fresh and crisp. Only at 10x baseline did the output break, and even then it broke into schema listing content (“This statement is an example of: Fact / Suggestion / Question / Prediction”) rather than tech Apple content. Bass v1 (“The bass was loud”) did the same, output stayed music/instrument focused, never fish.

The second pattern we called **Takeover** (9 of 11), covering crane, jam, coach, bat, bass v2, fans, club, net and key. On crane, injection gave us “*a majestic, almost life sized, imitation of the famous bird*” at 10x, a coherent bird description. On jam the model shifted to “*The repair work was done by a skilled technician*” at 5 10x. On coach we got “*the bus was packed with excited fans*” at 5x. On bass v2 (our matched pair test, discussed in Section 4.6) the takeover happened even at 1x. Table 6 has the full breakdown with sample generations.

Apple is uniquely resistant in that it breaks into non target schema content at 10x rather than either the target or the prompts original context, this might tell us something specific about how tech-Apple is stored (see Section 6 for caveats).

Table 6. Output-level response to target feature injection across 11 polysemes. Every injection produced a strong workspace effect (see Table 5), yet output generation responded differently: 2 polysemes resisted, 9 exhibited semantic takeover (1 partial). Apple is uniquely resistant in that at $\alpha=10\times$ the model breaks into schema-listing content rather than the target concept. The resistant cases share a structural property: their predicate is semantically incompatible with the injected sense (*a rotten company, a loud fish*); takeover cases have predicates that accept both senses.

Polyseme	Prompt (abbrev.)	Class	Output at pivotal α
apple	"...apple is rotten"	RESISTANT	schema break at 10x: “ <i>Fact / Suggestion / Question</i> ”
bass v1	"...bass was loud"	RESISTANT	stays music/instrument, never fish
bass v2	"...bass slipped from her grasp"	TAKEOVER	fish takeover at 1x: “ <i>a fat, silver beauty...she’d fought it</i> ”
crane	"...construction crane"	TAKEOVER	coherent bird at 10x: “ <i>a majestic, almost life-sized...bird</i> ”
jam	"...fixing the jam"	TAKEOVER	repair takeover at 5–10x: “ <i>repair work by a skilled technician</i> ”
coach	"...took them to the game"	TAKEOVER	bus takeover at 5x: “ <i>the bus was packed with excited fans</i> ”
bat	"...bat is quick"	TAKEOVER	baseball at 10x: “ <i>a good fielder...a good hitter</i> ”
fans	"...started making noise"	TAKEOVER	AC takeover at 10x: “ <i>the air conditioning system was on its last breath</i> ”
club	"...had to be replaced"	TAKEOVER	golf takeover at 10x: “ <i>the golf club was replaced after...golf tournament</i> ”
net	"...fixing the net"	TAKEOVER	fishing at 5x: “ <i>the fish were caught in the net</i> ”
key	"...change the key"	TAKEOVER (partial)	musical terms at 5–10x: “ <i>minor</i> ”, “ <i>flat</i> ”, “ <i>shift</i> ”

Summary: 2 **RESISTANT** (both share semantically-locking predicates), 9 **TAKEOVER** (including 1 partial).

4.6 The bass matched pair: controlled evidence

The bass matched pair is our strongest single piece of controlled evidence. Same noun (bass), same target feature (F718 fish), same injection protocol, we only changed the predicate. On “The bass was loud” (bass v1), F718 fires at baseline 0.00 and injection produces no fish content at any alpha, the output stays music/instrument focused. On “The bass slipped from her grasp” (bass v2), F718 fires at baseline 13.88 and injection produces

immediate fish takeover at 1x (“*The bass, a fat, silver beauty, had been her prize*”). The only variable is the predicate, “was loud” locks the sound sense in and rules out fish compatibility, “slipped from her grasp” is a predicate that works for both a fish and an instrument. This pins down predicate-semantic compatibility as what decides whether injection reaches output (Table 7).

Table 7. The bass matched-pair: same noun, same target feature (F718 fish), same injection protocol. The only variable is the predicate. The result inverts completely, from resistance to immediate takeover at $\alpha=1\times$, establishing predicate-semantic compatibility as the mechanism gating whether injection reaches the output layer. This is the paper’s cleanest single piece of causal evidence.

Prompt	Predicate compatibility	Baseline act. of F718	Class	Generation at $\alpha=1\times$
"The bass was loud."	<i>loud</i> incompatible with fish	0.00	RESISTANT	"The music was thumping. The crowd was packed..."
"The bass slipped from her grasp."	<i>slipped from grasp</i> compatible with fish	13.88	TAKEOVER	"The bass, a fat, silver beauty, had been her prize..."

4.7 Controls

Across all 11 polysemes and every alpha value both controls did what we expected. Random feature injection (F7000) kept the target features activation at 0.0, no target concept content showed up in any generation, and at $\alpha=10\times$ the outputs degenerated into repetitive loops (“by by by by...”). Gaussian noise did the same. Neither control produced target concept content in any polyseme (Table 8). The high alpha degeneration into loops is actually useful information, it confirms the residual stream was being perturbed by the injection at those magnitudes, just not in a meaningful direction. Only the target features direction produces the intended concept effect, which confirms direction-specificity is what drives the injection effect.

Table 8. Control-condition behaviour across all 11 polysemes and all α values. Random-feature injection (F7000, arbitrary unrelated SAE feature) and Gaussian noise (random unit vector, L_2 -matched to target direction) were run alongside every target injection. Neither control produced target-concept content in any polyseme at any α . High- α degeneration into repetitive loops confirms the residual stream was being perturbed, but only the target direction produced the intended concept effect, establishing direction-specificity.

Condition	Target F# activation	Target-concept content in output	Failure mode at $\alpha=10\times$
target (FF)	scales linearly with α	yes (in 9/11 polysemes)	concept-content or schema break (apple)
random-feature (F7000)	stays at 0.0	no (all 11)	repetitive loops (“by by by...”)
gaussian noise	stays at 0.0	no (all 11)	repetitive loops or garbage tokens

5. INTERPRETATION

5.1 The predicate compatibility mechanism

The resistant vs takeover split (2 vs 9) is not random, it likely comes down to whether the prompt’s predicate can accept the injected sense at all. The resistant cases (apple “is rotten”, bass v1 “was loud”) both have predicates that are semantically incompatible with the injected concept. Such as Apple, the company can not literally be organically “rotten” in the way fruit rots, and a fish isnt something that can be “loud” in the way an instrument or a sound can. The takeover cases (crane “watched”, jam “fixing”, coach “took them to the game” etc) all have predicates that either work for both senses of the polyseme or dont strongly commit to either, which is way more ideal than the semantically incompatible ones. “She watched” works for watching a bird or watching a construction crane, “fixing” works for fixing a printer jam or making a fruit jam, “took them to the game” works for a sports coach or a coach bus. The bass matched pair from Section 4.6 confirms this causally, with only the predicate changing the output response inverts entirely. Its possible also, that larger prompts exhibit more broker behaviour as multiple concepts or a complex task may use brokers in a more dynamic way to allow concepts to be held until output. This is purely theoretical, but we believe its likely that in longer prompts predicates are richer and more necessary to context. This is especially relevant for continuous task prompts, ie “do task a” + “edit task a” + “now explain the task at hand” where the input may rely on previously mentioned context “a” and not explicitly stated in every prompt “now explain task at hand” for example. This is not an extensive example, its just to illustrate our point here in a simple form.

5.2 Polysemanticity as a strategic tool

Putting all this together, our data supports a fairly strong claim about how polysemanticity operates in this model. It’s not just a passive artefact of how features get stored within its neurons, its actively used by the model as an inference tool, at least in our chosen Gemma model. When the model is uncertain about a prompt’s concept, specific broker features fire to hold alternative interpretations afloat as possibilities (Section 4.1, 4.2). These broker features can be causally invoked via injection (Section 4.3, 4.4), but whether the injected content actually reaches the output depends on a second mechanism at the output layer that gates by predicate compatibility (Section 4.5, 4.6). Both behaviours require the model to have polysemantic representation to work at all. A purely monosemantic model likely wouldn’t have the alternative senses available to broker between, so it wouldn’t need to elicit brokers, or it would just behave worse on ambiguous tasks, and wouldn’t need the output gating mechanism because there would be no injected alt sense content to filter in the first place because everything would be organised on scale into individual non overlapping concepts. This isn’t polysemanticity being tolerated as a bug, its polysemanticity being used.

5.3 Reconciliation with the superposition view

This finding doesn’t necessarily contradict the Elhage superposition view, we think both can be true at once. Superposition explains how polysemantic representation emerged in the first place, dimensional constraint in smaller models forced the model to pack multiple features per dimension. Our finding adds that even at scale, when dimensional pressure is easing as network space increases, this constraint solution has been either repurposed as functional infrastructure for handling ambiguity or that the large scale superposition was hiding it and other mechanisms that the model may have been using superposition for. Its not to say directly “the mainstream is wrong” so much as “the decomposition focused framing of superposition has missed a functional role that the same mechanism now uses”. Nailing this down more mechanistically would require testing whether purely monosemantic architectures can achieve equivalent output behaviour on ambiguous prompts without the broker infrastructure, we flag this as future work in Section 7.

6. LIMITATIONS AND NEGATIVE RESULTS

Here we want to flag the following limitations we encountered during the research as much as we can.

1st, this work uses a single 2B open model (Gemma-2-2B-IT), which is small compared to the frontier closed models used in Anthropic’s workspace paper, our findings may not scale directly into these models for numerous reasons.

2nd, we used a single SAE (Gemma Scope L20 16k) which has real feature coverage gaps, this can mainly be seen via the 5 of 11 new polysemes we tried failed target feature discovery entirely.

3rd, Neuronpedia auto interpretations aren’t always accurate, we saw two clear misclassifications in this study, F6936 was labelled “metal music bands” when we wanted the metal element sense of **lead**, and F7593 was the same sense weighting feature not an alt sense feature for **scale**.

4th, our sample size is small, 11 polysemes with clean injection data and only 2 clean resistance cases means the predicate compatibility claim would be likely shifted by a larger polyseme survey.

5th, broker discovery via Neuronpedia’s keyword search API is incomplete, querying “poultry” returned no L20 features despite F8002 being labelled “laying hens and poultry”, meaning our 12 undetermined polysemes are a possible methodological artefact rather than definitive evidence of broker absence.

6th, related to the previous point, the 73% broker rate refers only to the 11 polysemes we could definitively test, treating the 12 undetermined ones as non brokers gives a lower bound of 35%, the true rate lies likely somewhere between.

7th, J-lens works well at L20 but showed interpretation difficulties at earlier layers (L6, L13) where token gibberish is prominent, so our layer choice was empirically justified rather than principled from the start. Its also entirely possible some of the token gibberish acts as ambiguity brokers early on for other mechanisms related or not to concept inference, or form the precursors for brokers, although these would be hard to test under the time constraints which is why we focused on L20.

7. FUTURE WORK

7.1 Feature ablation experiments (necessity test)

The most obvious follow up is to test whether brokers are actually necessary by ablating them, ie the opposite of what we did with injection. Setting the broker features activation to zero via a hook, or subtracting the decoder direction from the residual stream, would tell us if the model breaks without the broker, tries to use something else, or just carries on unaffected. The infrastructure is already there from the injection experiments, we just flip the sign on the injection hook. This is arguably the single most valuable follow up we can do given how quickly it could be run.

7.2 Cross model replication

Everything in this paper was done on Gemma-2-2B-IT. Testing whether the broker mechanism replicates on Gemma-2-9B-IT, Llama-3-8B or similar would tell us if this is scale invariant or scale dependent. Cross model replication is standard in interpretability work and would need to be done before making any general claim about polysematicity as inference infrastructure.

7.3 Predicate compatibility predictions

Our predicate compatibility mechanism (Section 5.1) makes a testable prediction, we should be able to predict resistance vs takeover from the prompts predicate before we even run the injection. Constructing matched pairs where only the predicate varies (like our bass matched pair in Section 4.6) but doing this systematically across many polysemes would test the prediction properly.

7.4 Sense frequency correlation

One hypothesis we havent tested is that brokers might form only for polysemes where both senses have similar pretraining frequency. Correlating broker presence with n gram frequencies from a large corpus for each sense would test this. This might also help predict which polysemes will show brokers before running any injection at all.

7.5 Attempt monosemantic architecture comparison

The strongest possible test of “polysematicity is a functional tool” would be to compare our results against a purely monosemantic architecture on the same polyseme prompts. This is impractical for this study because of the compute cost of retraining a model with monosemantic constraints, but its a long term direction worth flagging.

7.6 Exhaustive SAE feature enumeration

We could iterate over all ~16,000 L20 SAE features, pull their Neuronpedia labels via the feature endpoint, filter for alt sense candidates per polyseme manually, then run activation checks. This bypasses Neuronpedia’s search API entirely, which we found returns no matches for many valid alt sense keywords (eg “poultry” fails to return F8002 despite F8002 being labelled “laying hens and poultry”). This method would resolve the 12 currently undetermined polysemes and give us a definitive broker rate rather than the 35 73% range we can report from the current methodology.

7.7 Multi feature census per polyseme

For the polysemes where we did find broker features (apple, crane, jam etc) we only tested one feature per concept. This means we cannot currently confirm if the different patterns we saw like apple appearing to have a pure broker pattern vs cranes content feature plus broker pattern, are actually different mechanisms in how the model represents each concept, or if we just happened to pick different types of features per concept. To

resolve this we would need to test all L20 SAE features that reference any concepts for each polyseme and then characterise their activation patterns across the ambig and disambig prompts, this would let us confirm if brokers come in genuine sub types, or, if we simply tested different features per concept, given more time this is something we would love to do as there may even be more interesting mechanisms here, or it could refute our findings as well as make them stronger.

8. DATA AND CODE

All code, cell outputs, and generation samples for this project will be published at <https://github.com/AntonGMorris/ambiguity-brokers> alongside the modified Jacobian lens variant we used (see Section 2.4). The specific model and tool artefacts referenced throughout the report are:

- Model: `google/gemma-2-2b-it` on HuggingFace
- Pre-fit Jacobian lens: `neuronpedia/jacobian-lens` on HuggingFace, artefact `gemma-2-2b-it/jlens/Salesforce-wikitext/gemma-2-2b-it_jacobian_lens.pt`
- SAE: Gemma Scope L20, `gemma-scope-2b-pt-res`, ID `layer_20/width_16k/average_l0_71`
- Feature interpretations retrieved via Neuronpedia’s public API at https://neuronpedia.org/api/feature/gemma-2-2b/20-gemmascope-res-16k/{feature_id}

The Colab notebook used to run all experiments will be made public in the same GitHub repository post-review.

9. REFERENCES

- Lindsey et al. (Anthropic, 2026). *The workspace paper*. <https://transformer-circuits.pub/2026/workspace/index.html>
- Elhage et al. (Anthropic). *Toy Models of Superposition*. https://transformer-circuits.pub/2022/toy_model/index.html
- Anthropic (2026). *Jacobian Lens open source code*. <https://github.com/anthropics/jacobian-lens>
- Google DeepMind. *Gemma Scope: Sparse Autoencoders on Gemma-2*.
- Neuronpedia. <https://neuronpedia.org>
- Baars, B. (1988). *A Cognitive Theory of Consciousness*. Cambridge University Press.
- Nakaishi et al. (2025). *Rethinking Evaluation of Sparse Autoencoders through the Representation of Polysemous Words*. <https://arxiv.org/abs/2501.06254>
- Anonymous (2025). *Start Making Sense(s): A Developmental Probe of Attention Specialization Using Lexical Ambiguity*. <https://arxiv.org/abs/2511.21974>
- Xu et al. (2025). *Do Language Models Encode Semantic Relations? Probing and Sparse Feature Analysis*. <https://arxiv.org/abs/2603.17624>

APPENDIX

Neuronpedia feature IDs and labels

All features referenced in this paper are from Gemma Scope `layer_20/width_16k/average_l0_71`. Labels are Neuronpedia auto-interpretations. Feature URLs follow the pattern <https://neuronpedia.org/gemma-2-2b/20-gemmascope-res-16k/{feature-id}>.

- F16331: Apple, iOS, iPhone content (apple broker)
- F8002: laying hens and poultry (crane broker / bird sense)
- F14694: baseball (bat broker)
- F718: fish names and schools (bass v1 / bass v2 broker)
- F9643: repair and maintenance (jam broker)
- F6316: fruit (jam alt-sense scan feature)

- F8027: bus and train transport (coach broker)
- F11413: air movement and quality (fans, discovered feature, non-broker)
- F9675: golf and associated items (club broker)
- F6460: fishing tackle (net broker)
- F8051: major and minor musical keys (key, discovered feature, non-broker)
- F7593: measuring physical weight (scale, excluded: same-sense feature)
- F6936: metal music bands (lead, excluded: semantically mismatched)
- F7000: random control feature (L2-matched injection control)